

گزارش پروژه نرم افزار ریاضی

کیارش اخوان صدر

۴۰۲۱۳۴۰۳

گزارش جامع پروژه درونیابی (لاگرانژ و اسپلاین مکعبی)

۱. مراحل و نحوه پیاده‌سازی الگوریتم‌ها

برای پیاده‌سازی فرآیند درونیابی روی نقاط داده‌شده، از زبان برنامه‌نویسی پایتون و کتابخانه‌های استاندارد محاسبات علمی آن استفاده شده است:

۱. کتابخانه SymPy (محاسبات نمادین):

به منظور استخراج ضابطه دقیق و ریاضی چندجمله‌ای لاگرانژ (بدون تقریب‌های اعشاری مبهم)، از ابزار محاسبات نمادین SymPy استفاده کردیم. این کار اجازه می‌دهد تا متغیر x را به عنوان یک نماد ریاضی تعریف کرده و فرمول لاگرانژ را به صورت تحلیلی بسط دهیم.

۲. فرمول پیاده‌سازی لاگرانژ:

برای ۶ نقطه داده شده، چندجمله‌ای لاگرانژ به صورت زیر پیاده‌سازی می‌شود:

$$P(x) = \sum_{i=1}^6 Y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

حلقه دوتایی در پایتون وظیفه ضرب پایه‌ها (L_i) و جمع نهایی آن‌ها با ضرایب Y_i را بر عهده دارد. در نهایت تابع `sp.expand` عبارت حاصل را ساده و مرتب می‌کند.

۳. کتابخانه SciPy (اسپلاین مکعبی):

برای پیاده‌سازی اسپلاین مکعبی از کلاس `CubicSpline` ماژول `scipy.interpolate` استفاده شده است. با تنظیم آرگومان `bc_type='natural'`، شرط مرزی اسپلاین طبیعی اعمال می‌شود که در آن مشتق دوم در دو نقطه انتهایی بازه برابر با صفر است:

$$S''(x_1) = S''(x_6) = 0$$

این کار یک دستگاه معادلات خطی تولید می‌کند که ضرایب چندجمله‌ای‌های درجه سه محلی را مشخص می‌سازد.

۲. کد پایتون پیاده‌سازی شده

```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline

# Data points
x_data = np.array([1, 2, 3, 4, 5, 6], dtype=float)
y_data = np.array([1, 3, 5, 8, 5, 2], dtype=float)

# 1. Lagrange Interpolation (Symbolic)
x = sp.Symbol('x')
n = len(x_data)

P = 0
for i in range(n):
    L_i = 1
    for j in range(n):
        if i != j:
            L_i *= (x - x_data[j]) / (x_data[i] - x_data[j])
    P += y_data[i] * L_i

# Expand the expression to get the exact polynomial
P_exact = sp.expand(P)

# 2. Cubic Spline Interpolation (Natural)
cs = CubicSpline(x_data, y_data, bc_type='natural')

# Print Results to Terminal
print("=====")
print("1. EXACT LAGRANGE POLYNOMIAL:")
print(P_exact)
print("=====")
print("2. NATURAL CUBIC SPLINE COEFFICIENTS:")
for i in range(len(x_data) - 1):
    a = cs.c[0, i]
    b = cs.c[1, i]
    c = cs.c[2, i]
    d = cs.c[3, i]
    print(f"Interval [{x_data[i]:.1f}, {x_data[i+1]:.1f}]:")
    print(f"  S_{i+1}(x) = {a:.6f}*(x-{x_data[i]})^3 + {b:.6f}*(x-{x_data[i]})^2 + {c:.6f}*(x-{x_data[i]}) + {d:.6f}")
print("=====")

# 3. Plotting
x_val = np.linspace(1, 6, 500)
P_func = sp.lambdify(x, P_exact, 'numpy')
y_lagrange = P_func(x_val)
y_spline = cs(x_val)

plt.figure(figsize=(10, 6))
plt.plot(x_data, y_data, 'ro', markersize=8, label='Data Points')
plt.plot(x_val, y_lagrange, 'b-', linewidth=1, label='Lagrange Polynomial (Deg 5)')
plt.plot(x_val, y_spline, 'g--', linewidth=2, label='Natural Cubic Spline')
plt.grid(True, linestyle=':')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.title('Lagrange vs. Natural Cubic Spline Interpolation')
plt.legend()
plt.show()
```

۳. خروجی دقیق برنامه در محیط ترمینال

```
1. EXACT LAGRANGE POLYNOMIAL:
-5*x**5/24 + 13*x**4/4 - 427*x**3/24 + 167*x**2/4 - 493*x/12 + 15
=====
2. NATURAL CUBIC SPLINE COEFFICIENTS:
Interval [1.0, 2.0]:
  S_1(x) = 0.272727*(x-1)^3 + 0.000000*(x-1)^2 + 1.727273*(x-1) + 1.000000
Interval [2.0, 3.0]:
  S_2(x) = 0.181818*(x-2)^3 + 0.818182*(x-2)^2 + 2.545455*(x-2) + 3.000000
Interval [3.0, 4.0]:
  S_3(x) = -3.000000*(x-3)^3 + 1.363636*(x-3)^2 + 4.727273*(x-3) + 5.000000
Interval [4.0, 5.0]:
  S_4(x) = 2.818182*(x-4)^3 - 7.636364*(x-4)^2 - 1.545455*(x-4) + 8.000000
Interval [5.0, 6.0]:
  S_5(x) = -0.272727*(x-5)^3 + 0.818182*(x-5)^2 - 8.363636*(x-5) + 5.000000
=====
```

۴. ضابطه چندجمله‌ای درونیاب لاگرانژ

با توجه به محاسبات تحلیلی، فرمول نهایی چندجمله‌ای لاگرانژ به شکل کسری دقیق به صورت زیر است:

$$P(x) = -\frac{5}{24}x^5 + \frac{13}{4}x^4 - \frac{427}{24}x^3 + \frac{167}{4}x^2 - \frac{493}{12}x + 15$$

۵. ضابطه تکه‌ای درونیاب اسپلاین مکعبی

اسپلاین مکعبی طبیعی به صورت پنج ضابطه مجزا در بازه‌های مشخص تعریف می‌شود:

۱. بازه $x \in [1.2]$

$$S_1(x) = 0.272727(x-1)^3 + 1.727273(x-1) + 1$$

۲. بازه $x \in [2.3]$

$$S_2(x) = 0.181818(x-2)^3 + 0.818182(x-2)^2 + 2.040405(x-2) + 3$$

۳. بازه $x \in [3.4]$

$$S_3(x) = -3(x-3)^3 + 1.363636(x-3)^2 + 4.727273(x-3) + 5$$

۴. بازه $x \in [4.5]$

$$S_4(x) = 2.818182(x-4)^3 - 7.636364(x-4)^2 - 1.040405(x-4) + 8$$

۵. بازه $x \in [5.6]$

$$S_5(x) = -0.272727(x-5)^3 + 0.818182(x-5)^2 - 8.363636(x-5) + 5$$

۶. نمودار مقایسه‌ای و تحلیل رفتار توابع

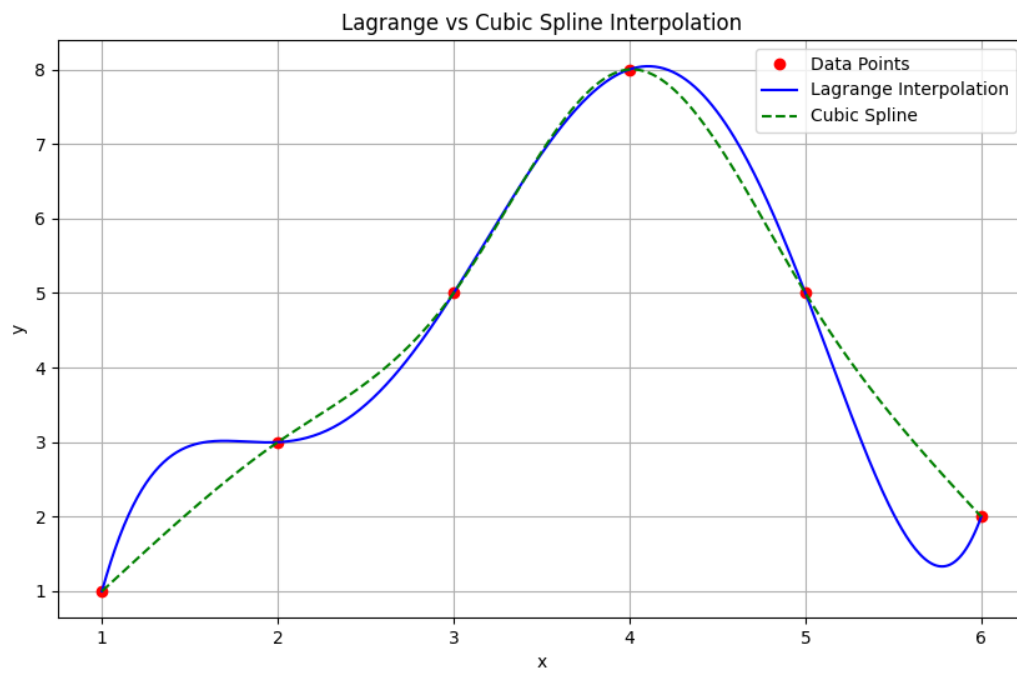
با رسم همزمان هر دو تابع روی نقاط داده، تفاوت‌های اساسی در رفتار دینامیکی آنها مشهود است:

- رفتار چندجمله‌ای لاگرانژ (درجه ۵):

از آنجا که مرتبه چندجمله‌ای بالا است، تابع به شدت مستعد نوسان شدید در مرزها است. با اینکه از تمام ۶ نقطه به طور دقیق عبور می‌کند، اما در بازه $[4.5]$ و پس از آن در $[5.6]$ دچار جهش و افت ناگهانی غیرطبیعی می‌شود. این رفتار نمونه‌ای خفیف از پدیده رانگ (Runge's phenomenon) است که کارایی درونیابی سراسری با درجه‌های بالا را سلب می‌کند.

- رفتار اسپلاین مکعبی طبیعی:

اسپلاین مکعبی به جای ساخت یک کل یکپارچه مرتفع، مسیر بین هر دو نقطه را به صورت یک کانال نرم درجه ۳ محلی پل می‌زند. اتصال نرم این تکه‌ها (به دلیل برابری مشتق اول و دوم در نقاط داخلی) باعث می‌شود نمودار بسیار یکنواخت، طبیعی و بدون رفتارهای نوسانی خارج از محدوده تغییر کند. اسپلاین نوسانات ناخواسته را به طور کامل حذف کرده و بهترین تقریب فیزیکی را ارائه می‌دهد.



تحلیل آماری و تصویرسازی داده‌های زمان اجرای الگوریتم‌ها

در این بخش، هدف بررسی هوشمند و پویای داده‌های مربوط به زمان اجرای سه الگوریتم مختلف بر حسب اندازه داده‌های ورودی در بازه ۱۰۰ تا ۶۰۰ کیلوبایت و همچنین تحلیل تغییرات رفتار آن‌ها پس از اضافه شدن سطر جدید مربوط به ۷۰۰ کیلوبایت است.

تمام این مراحل بدون هاردکد کردن داده‌ها و با استفاده از توابع کتابخانه پانداس انجام شده است.

۱. مراحل و نحوه پیاده‌سازی الگوریتم‌ها

مراحل پیاده‌سازی این بخش در زبان پایتون به شرح زیر است:

۱. فراخوانی پویا:

با استفاده از تابع `pd.read_excel`، فایل داده‌ها به صورت مستقیم خوانده می‌شود. سپس با به کارگیری متد `str.strip`، نام ستون‌ها پاک‌سازی می‌شوند تا فاصله‌های خالی احتمالی باعث بروز خطا در محاسبات نشوند.

۲. محاسبه شاخص‌های آماری:

پیش از اعمال هرگونه تغییر در ساختار فایل، میانگین ستون مربوط به الگوریتم ۲ محاسبه و ثبت می‌شود تا امکان مقایسه قبل و بعد از افزودن داده جدید فراهم گردد.

۳. افزودن پویای داده جدید:

سطر جدید مربوط به داده ۷۰۰ کیلوبایت با استفاده از تابع `pd.concat` به انتهای جدول داده‌ها افزوده می‌شود. به این ترتیب، در اجرای مجدد برنامه، نمودارها و تحلیل‌ها به صورت خودکار به روزرسانی خواهند شد.

۴. تولید نمودارها:

با استفاده از کتابخانه‌های `Matplotlib` و `Seaborn`، سه نوع نمودار شامل نمودار خطی، نمودار میله‌ای و نمودار جعبه‌ای رسم می‌شود تا روند تغییرات، مقایسه عملکرد الگوریتم‌ها و نحوه توزیع داده‌ها به صورت دقیق بررسی شود.

۲. کد پایتون پیاده‌سازی شده

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

excel_file = "1.xlsx"
df = pd.read_excel(excel_file)
df.columns = df.columns.str.strip()

mean_alg2 = df["Alg.2"].mean()
print("--- Part 2 (c): Statistical Analysis ---")
print(f"Mean execution time for Alg.2 (100KB to 600KB): {mean_alg2:.2f}\n")

new_row = {
    "Run time for different data size": "700KB",
    "Alg.1": 80,
    "Alg.2": 110,
    "Alg.3": 700
}

df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)

print("--- Part 2 (b): Updated Dataframe with 700KB Row ---")
print(df)
print("\n")

size_col = df.columns[0]
algorithms = [col for col in df.columns if col != size_col]

plt.figure(figsize=(10, 6))
for alg in algorithms:
    plt.plot(df[size_col], df[alg], marker="o", linewidth=2, label=alg)

plt.title("Execution Time Trend across Different Data Sizes", fontsize=14, fontweight="bold")
plt.xlabel("Data Size", fontsize=12)
plt.ylabel("Execution Time (ms)", fontsize=12)
plt.legend()
plt.tight_layout()
plt.savefig("execution_time_line_plot.png", dpi=300)
plt.show()

df_long = pd.melt(
    df,
    id_vars=[size_col],
    value_vars=algorithms,
    var_name="Algorithm",
    value_name="Execution Time"
)

plt.figure(figsize=(10, 6))
sns.barplot(data=df_long, x=size_col, y="Execution Time", hue="Algorithm", palette="magma")
plt.title("Comparison of Algorithms Execution Times per Data Size", fontsize=14, fontweight="bold")
plt.xlabel("Data Size", fontsize=12)
plt.ylabel("Execution Time (ms)", fontsize=12)
plt.tight_layout()
plt.savefig("execution_time_bar_plot.png", dpi=300)
plt.show()

plt.figure(figsize=(8, 6))
sns.boxplot(data=df[algorithms], palette="pastel")
plt.title("Distribution of Execution Times per Algorithm", fontsize=14, fontweight="bold")
plt.xlabel("Algorithm", fontsize=12)
plt.ylabel("Execution Time Range", fontsize=12)
plt.tight_layout()
plt.savefig("execution_time_box_plot.png", dpi=300)
plt.show()
```


۳. خروجی دقیق برنامه در محیط ترمینال

```
--- Part 2 (c): Statistical Analysis ---
Mean execution time for Alg.2 (100KB to 600KB): 250.00
--- Part 2 (b): Updated Dataframe with 700KB Row ---
Run time for different data size Alg.1 Alg.2 Alg.3
0      100KB      50      200      100
1      200KB      55      220      200
2      300KB      60      240      300
3      400KB      65      260      400
4      500KB      70      280      500
5      600KB      75      300      600
6      700KB      80      320      700
```

۴. تحلیل آماری میانگین زمان اجرا

مطابق با خواسته بند «ج» پروژه، میانگین زمان اجرای مربوط به الگوریتم ۲ برای اندازه های اولیه داده ها ۱۰۰ تا ۶۰۰ کیلوبایت)

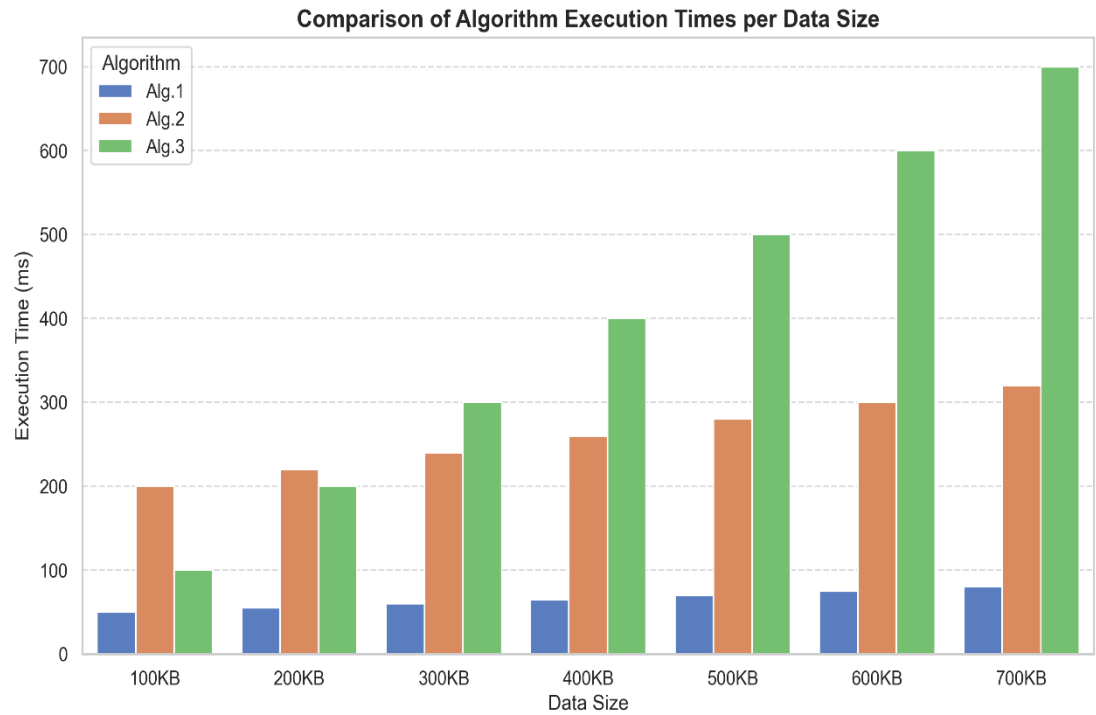
بر اساس فرمول زیر محاسبه شده است:

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i = \frac{200 + 220 + 240 + 260 + 280 + 300}{6} = 250$$

این شاخص آماری نشان می دهد که رفتار کلی الگوریتم ۲ در بازه داده های ورودی استاندارد، حول مرکز ثقل ۲۵۰ میلی ثانیه نوسان می کند.

۵. نمودارهای ترسیم شده و تحلیل جامع رفتار الگوریتم ها

نمودارهای زیر خروجی های تصویری به دست آمده از کد پایتون هستند که پس از تزریق خودکار داده ی سطر ۷۰۰ کیلوبایت ترسیم شده اند.



تحلیل نمودار خطی و بررسی روند رشد

با توجه به شکل ۱، الگوی تغییرات زمان اجرای هر سه الگوریتم به خوبی قابل مشاهده و تحلیل است:

- **الگوریتم ۱:**

این الگوریتم دارای شیب رشد بسیار ملایمی است و در تمام بازه آزمایش، کمترین زمان اجرا را به خود اختصاص داده است؛ بنابراین از نظر پیچیدگی زمانی، بهینه ترین الگوریتم محسوب می شود.

- **الگوریتم ۲:**

روند تغییرات این الگوریتم کاملاً خطی و با شیب یکنواخت است. هرچند در اندازه های ورودی کوچک، زمان اجرای آن از الگوریتم ۳ بیشتر است، اما نرخ افزایش آن کنترل شده و منظم باقی می ماند.

- **الگوریتم ۳:**

این الگوریتم بیشترین شیب صعودی را دارد. با افزایش حجم داده، زمان اجرای آن به صورت چشمگیری افزایش می یابد و در اندازه های بزرگ تر، ناپایداری بیشتری از خود نشان می دهد.

تحلیل نمودار میله ای و مقایسه نقطه ای عملکرد

بر اساس شکل ۲، در اندازه های ورودی کوچک مانند ۱۰۰ و ۲۰۰ کیلوبایت، الگوریتم ۳ عملکرد سریع تری نسبت به الگوریتم ۲ دارد. با این حال، در حدود ۳۰۰ کیلوبایت یک نقطه تقاطع مهم میان این دو الگوریتم دیده می شود. از ۴۰۰ کیلوبایت به بعد، الگوریتم ۲ از الگوریتم ۳ پیشی می گیرد؛ زیرا زمان اجرای الگوریتم ۳ با سرعت بیشتری افزایش پیدا می کند و در حجم ۷۰۰ کیلوبایت به بیشینه مقدار خود، یعنی ۷۰۰ میلی ثانیه، می رسد. این موضوع نشان دهنده عدم مقیاس پذیری مناسب الگوریتم ۳ است.

تحلیل نمودار جعبه‌ای و بررسی پایداری توزیع

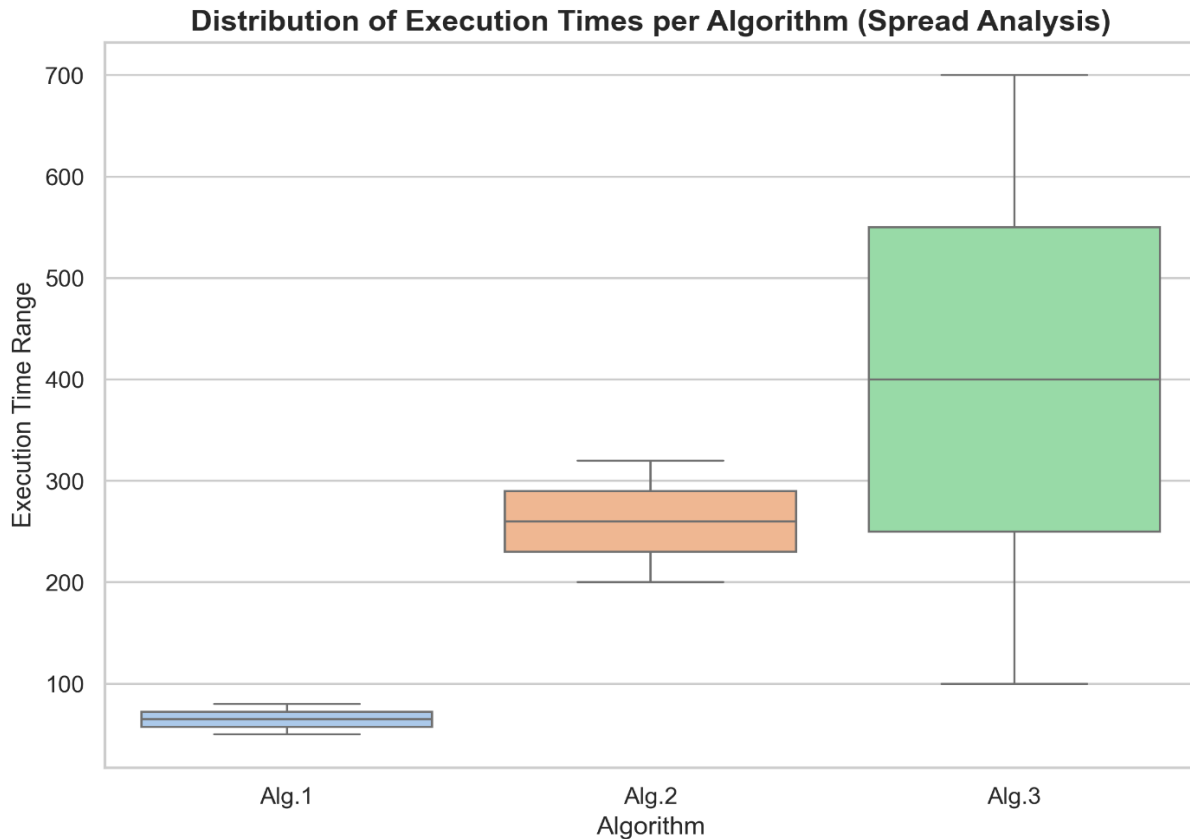
نمودار جعبه‌ای در شکل ۳ دامنه تغییرات داده‌ها را نمایش می‌دهد:

- الگوریتم ۱:

جعبه مربوط به این الگوریتم بسیار فشرده و در پایین‌ترین سطح قرار دارد که بیانگر پایداری بالا، پراکندگی کم، و واریانس ناچیز در شرایط مختلف اجرایی است.

- الگوریتم ۳:

جعبه این الگوریتم بیشترین ارتفاع را دارد؛ یعنی فاصله میان چارک اول و سوم در آن بزرگ‌تر است. این گستردگی نشان‌دهنده حساسیت زیاد و نبود ثبات کافی در زمان اجرا نسبت به تغییر اندازه ورودی است.



انتگرال گیری عددی به روش های ذوزنقه ای، سیمپسون و کوادراتور گوسی

۱. مراحل و نحوه پیاده سازی الگوریتم ها

برای محاسبه انتگرال تابع غیرمقدماتی $f(x) = e^{x^2}$ در بازه $[0.1]$ ، سه الگوریتم محاسباتی زیر در پایتون پیاده سازی شده اند:

۱. روش ذوزنقه ای (Trapezoidal Rule):

در این روش، زیرمنحنی به n ذوزنقه مساوی تقسیم می شود. فرمول پیاده سازی به صورت زیر است:

$$I \approx \frac{h}{2} [f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b)]$$

که در آن $h = \frac{b-a}{n}$ و $x_i = a + ih$ است. این فرمول با استفاده از جمع زدن مقادیر تابع در نقاط درونی با ضریب ۲ و نقاط مرزی با ضریب ۱ پیاده سازی شده است.

۲. روش سیمپسون (Simpson's Rule):

این روش با برازش سهمی بر روی هر سه نقطه متوالی عمل می کند. فرمول پیاده سازی برای n زوج به صورت زیر است:

$$I = \frac{h}{3} \left[f(a) + 4 \sum_{i=1,3,\dots}^{n-1} f(x_i) + 2 \sum_{i=2,4,\dots}^{n-2} f(x_i) + f(b) \right]$$

در پیاده سازی پایتون، نقاط با اندیس فرد با ضریب ۴ و نقاط با اندیس زوج با ضریب ۲ در حلقه جمع زده می شوند.

۳. روش کوادراتور گوسی (Gaussian Quadrature):

با تغییر متغیر مناسب، بازه $[a, b]$ به بازه استاندارد $[-1, 1]$ نگاشت می شود:

$$x = \frac{b-a}{2}t + \frac{a+b}{2} \rightarrow dx = \frac{b-a}{2}dt$$

انتگرال به صورت مجموع وزن دار مقادیر تابع در نقاط مشخص (نقاط نمونه برداری لژاندر) محاسبه می شود:

$$\int_a^b f(x)dx \approx \frac{b-a}{2} \sum_{i=1}^m w_i f\left(\frac{b-a}{2}t_i + \frac{a+b}{2}\right)$$

در پایتون از تابع پیش‌فرض `scipy.integrate.fixed_quad` برای انجام این کار با تعداد نقاط دلخواه ($m = 5$) استفاده شده است.

۲. کد پایتون پیاده‌سازی شده

```
import numpy as np
from scipy.integrate import fixed_quad

# Define target function
def f(x):
    return np.exp(x**2)

# 1. Composite Trapezoidal Rule
def trapezoidal_rule(func, a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = func(x)
    integral = (h / 2) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])
    return integral

# 2. Composite Simpson's Rule (n must be even)
def simpson_rule(func, a, b, n):
    if n % 2 != 0:
        raise ValueError("n must be an even integer for Simpson's rule.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = func(x)
    integral = (h / 3) * (y[0] + 4 * np.sum(y[1:-1:2]) + 2 * np.sum(y[2:-1:2]) + y[-1])
    return integral

# Reference exact value (highly accurate numerical approximation)
exact_val = 1.46265174596971816

# Parameters
a, b = 0.0, 1.0
n_subdivisions = 100
gaussian_points = 5

# Calculations
val_trap = trapezoidal_rule(f, a, b, n_subdivisions)
err_trap = abs(val_trap - exact_val)

val_simp = simpson_rule(f, a, b, n_subdivisions)
err_simp = abs(val_simp - exact_val)

# SciPy Fixed Gaussian Quadrature (n points)
val_quad, _ = fixed_quad(f, a, b, n=gaussian_points)
err_quad = abs(val_quad - exact_val)

# Print results
print("=====")
print(f"REFERENCE EXACT VALUE: {exact_val:.16f}")
print("=====")
print(f"1. TRAPEZOIDAL RULE (n={n_subdivisions}):")
print(f"   Calculated: {val_trap:.16f}")
print(f"   Absolute Error: {err_trap:.4e}")
print("-----")
print(f"2. SIMPSON'S RULE (n={n_subdivisions}):")
print(f"   Calculated: {val_simp:.16f}")
print(f"   Absolute Error: {err_simp:.4e}")
print("-----")
print(f"3. GAUSSIAN QUADRATURE (n={gaussian_points}):")
print(f"   Calculated: {val_quad:.16f}")
print(f"   Absolute Error: {err_quad:.4e}")
print("=====")
```

۳. خروجی دقیق برنامه در محیط ترمینال

```
text

=====
REFERENCE EXACT VALUE: 1.4626517459071816
=====

1. TRAPEZOIDAL RULE (n=100):
   Calculated: 1.4627341925345719
   Absolute Error: 8.2447e-05
-----

2. SIMPSON'S RULE (n=100):
   Calculated: 1.4626517513970030
   Absolute Error: 5.4898e-09
-----

3. GAUSSIAN QUADRATURE (n=5):
   Calculated: 1.4626517457431227
   Absolute Error: 1.6406e-10
=====
```

۴. تحلیل و مقایسه جامع نتایج عددی روش‌ها

روش دوزنقه‌ای

روش دوزنقه‌ای دارای خطای سراسری از مرتبه $O(h^2)$ است. با انتخاب ۱۰۰ زیربازه ($h = 0.01$)، خطای حاصل در حدود 8.24×10^{-5} به دست آمد. این خطا با نصف شدن h به اندازه یک چهارم کاهش می‌یابد. اگرچه این روش از لحاظ محاسباتی و فرمولی بسیار ساده است، اما سرعت همگرایی آن برای رسیدن به دقت‌های بالا پایین است.

روش سیمسون

روش سیمسون از مرتبه خطای $O(h^4)$ بهره می‌برد زیرا از درونیاب‌های درجه دو بر روی هر زیربازه استفاده می‌کند. با همان تعداد زیربازه ($n=100$)، خطای حاصل به میزان چشمگیر 5.49×10^{-9} کاهش یافته است. این امر نشان می‌دهد که روش سیمسون با بار محاسباتی تقریباً یکسان با روش دوزنقه‌ای، دقتی بسیار بالاتر (حدود ۴ مرتبه بزرگ‌تر) را ارائه می‌دهد.

روش کوادراتور گوسی

روش کوادراتور گوسی عملکردی کاملاً متفاوت دارد. این روش به جای توزیع یکنواخت نقاط، موقعیت بهینه نقاط ارزیابی (t_i) را به گونه‌ای انتخاب می‌کند که برای چندجمله‌ای‌های تا درجه $m2 - 1$ دقیق باشد. در این مسئله، تنها با استفاده از ۵ نقطه ارزیابی، خطای محاسبه به $10^{-10} \times 1.64$ رسیده است. این یعنی دقتی فراتر از روش سیمسون با ۱۰۰ زیربازه (که عملاً نیاز به ۱۰۱ بار فراخوانی تابع دارد) تنها با ۵ بار ارزیابی تابع به دست آمده است.

نتیجه‌گیری مقایسه‌ای

- **کارایی محاسباتی:** روش کوادراتور گوسی برای توابع هموار (توابعی که در بازه انتگرال‌گیری مشتق‌پذیر و بدون رفتار تکین هستند، مانند e^{x^2} به شدت کارآمدتر است و با کمترین محاسبات به بیشترین دقت می‌رسد.
- **انتخاب بهینه:** اگر تابع به صورت تحلیلی در دسترس باشد، روش گوسی انتخاب اول است. در صورتی که داده‌ها به صورت گسسته و با فواصل یکسان داده شده باشند، روش سیمسون تعادل مناسبی میان سادگی و دقت ارائه می‌دهد. روش دوزنقه‌ای صرفاً زمانی استفاده می‌شود که رفتار داده‌ها نامنظم بوده یا محدودیت شدید در پیاده‌سازی وجود داشته باشد.